



ADR-010: API-first route table that generates OpenAPI

TASCO Insurance × VETC Growth Platform — Architecture Decision Record

iorta TechNXT

Version 1.0 · 07 October 2026

Contents

Context and problem statement

Decision drivers

Considered options

Decision outcome

Pros and cons of the options

Document	Version	Date	Owner	Status
ADR-010	1.0	07 October 2026	iorta TechNXT Architecture	Accepted

- **Related:** [ADR-002, Integration Architecture](#), section 5, [Security Architecture](#), section 6

Context and problem statement

Many consumers depend on the API:

- the console SPA and the customer app;
- partners (banks, showrooms, agents, fleets, inspection centres) integrating server to server;
- VETC and TASCO integration teams;
- security testers (DAST needs a spec).

API documentation that drifts from code causes integration defects and security blind spots, such as undocumented endpoints or missing auth.

Decision drivers

- One source of truth for path, method, authentication, permission and input schema.
- Authorisation must be impossible to forget on a new route.
- The published contract must match actual behaviour.

Considered options

1. **Routes as data** (`src/adapters/http/routes.js`). Each entry declares `method`, `path`, `auth`, `perm`, `scope`, `body`, `query`, `idempotent`, `loginLimited`, `demoOnly`, `rateCost`, `tag`, `summary`, `handler`. The same table drives the router, the pipeline (`auth` → `RBAC` → `validation`) and `openapi.js`.
2. Design-first OpenAPI YAML with code generation.
3. Framework annotations / decorators.

Decision outcome

Chosen option: **1.**

- `app.js` enforces each declaration in order:
 1. rate limit;
 2. body read (1 MiB cap, `application/json` only, objects only); malformed path parameters return 400;
 3. `authenticate(route.auth)`;
 4. `access.require(perm)`, plus the partner key `scope` where declared;
 5. `validate(query/body)`, which rejects unknown fields;
 6. the Idempotency-Key format check;
 7. the handler.
- `openapi.js` emits OpenAPI 3.1:
 - `security` (`bearerAuth` or `partnerApiKey`), `x-permission` and `x-audience` per operation;
 - `additionalProperties: false` request schemas, plus the `Idempotency-Key` header where required;
 - a uniform error schema `{error: {code, message, details, requestId}}`.

- The spec is served at `/api/openapi.json` and written to `docs/api/openapi.json` by `npm run job -- openapi`.
- There are 80 operations: 6 public, 60 staff, 10 customer and 4 partner. Every non-public route requires authentication. Staff routes declare `perm`, except `auth/logout`, `auth/me` and `auth/password`, which are self-service.

Consequences

- Good: the contract cannot drift, and DAST (ZAP) can import the generated spec in CI.
- Good: security review is simple. A reviewer reads one table to see every endpoint's exposure.
- Bad: **response schemas are generic** (`type: object`). **Fix:** add optional `response` schemas per route for partner-facing endpoints first.
- Bad: ABAC checks and ownership checks live inside handlers or services and are not visible in the table. Examples: `quote.partnerId === principal.partnerId` and `access.check(...)`. **Fix:** add a declarative `owner / abac` field so the review checklist covers them.
- Note: `/api/openapi.json` and `/api/meta` are public. The spec reveals the staff API surface. This is acceptable (no security through obscurity), but the production ingress may restrict the spec to internal networks.
- **API versioning:** the partner API is versioned in the path (`/api/partner/v1`). Internal APIs are versioned with the SPA (same deployable). Breaking partner changes require `v2` running alongside `v1` for at least 6 months.

Pros and cons of the options

Design-first YAML. Strong for external contracts. However, there are two artefacts to keep in sync, unless code generation adds dependencies.

Annotations. Concise. However, they need a framework, which conflicts with [ADR-002](#).